Fourteenth Implementation Iteration: Space Allocation

with Structured User Roles and UserRoleRule

Software Design / Iteration Report

Name: Najib Attar
Date: April 2026

1. Iteration Goal

The goal of this fourteenth implementation iteration is to improve the user model and introduce role-based access control for space requests. In previous iterations, the user role was represented as a plain string and did not directly affect allocation decisions.

In this iteration, the system introduces a structured `UserRole` enumeration and a new rule called `UserRoleRule`. This allows the system to make allocation decisions based on the requester's role. As a result, user roles now have direct behavioral meaning in the allocation process.

2. Changes from Iteration 13

Compared with the thirteenth iteration, the following changes were introduced:

- Replaced the plain string-based user role with a structured `UserRole` enumeration
- Updated the `User` model so that each user stores a typed role
- Updated CSV data loading so that role strings are converted into `UserRole` values
- Added a new rule: `UserRoleRule`
- Integrated `UserRoleRule` into the rule engine facade
- Updated the main test scenarios so that different users with different roles are evaluated

3. Scenario

The system loads users from file, where each user is assigned a structured role such as `Student`, `Instructor`, `Staff`, or `Administrator`. When a request is submitted, the rule

engine now checks not only capacity, feature, status, location, and time availability, but also whether the requester's role is authorized to request the selected space type.

For example, an instructor may request a laboratory, while a student cannot. A staff user may request a meeting room, while an administrator may request any supported space type.

4. Modules and Their Tasks

A. User Management Module

- Represent each user with a structured `UserRole`
- Eliminate ambiguity caused by raw string-based role values
- Provide role information to the request and rule evaluation process

B. Data Loading Module

- Read user role values from external CSV files
- Convert textual role values into `UserRole` enumeration values
- Construct `User` objects using the structured role type

C. Rule Evaluation Module

- Apply existing rules through the rule engine and facade
- Apply the new `UserRoleRule` to determine whether a requester is allowed to request a specific space type
- Return failure information when the requester role is not authorized

D. Allocation Management Module

- Continue processing one-time and recurring requests through the allocation service
- Use the updated rule engine result to approve or reject requests
- Create allocations only if all rules, including role authorization, pass

5. Role Authorization Policy

In this iteration, the following simple authorization policy is used:

- `Student` can request:
  - `Classroom`
  - `MeetingRoom`
- `Instructor` can request:
  - `Classroom`
  - `Laboratory`
  - `MeetingRoom`
- `Staff` can request:
  - `MeetingRoom`
- `Administrator` can request:
  - all supported space types

6. Iteration Comparison

Iter.	**Requests**	**Resources**	**Rules / Main Design Change**
1–4	OTR Only	Class → Lab → Meet	Availability + Cap.
5–6	OTR + RR	Class + Lab + Meet	(Redesign)
7	OTR + RR	Class + Lab + Meet	+ Feature
8–9	OTR + RR	Class + Lab + Meet	+ Status
10–11	OTR + RR	Class + Lab + Meet	+ Location
12	Abstract Request	Class + Lab + Meet	Polymorphism
13	Abstract Reques	Class + Lab + Meet	Rule Engine + Facade
14	Abstract Reques	Class + Lab + Meet	Structured User Roles + `UserRoleRule`

7. Updated Class Diagram

Figure 1 shows the updated simplified class diagram for this iteration. The diagram highlights the introduction of `UserRole` and `UserRoleRule`, and shows how role-based authorization is now integrated into the rule engine structure.

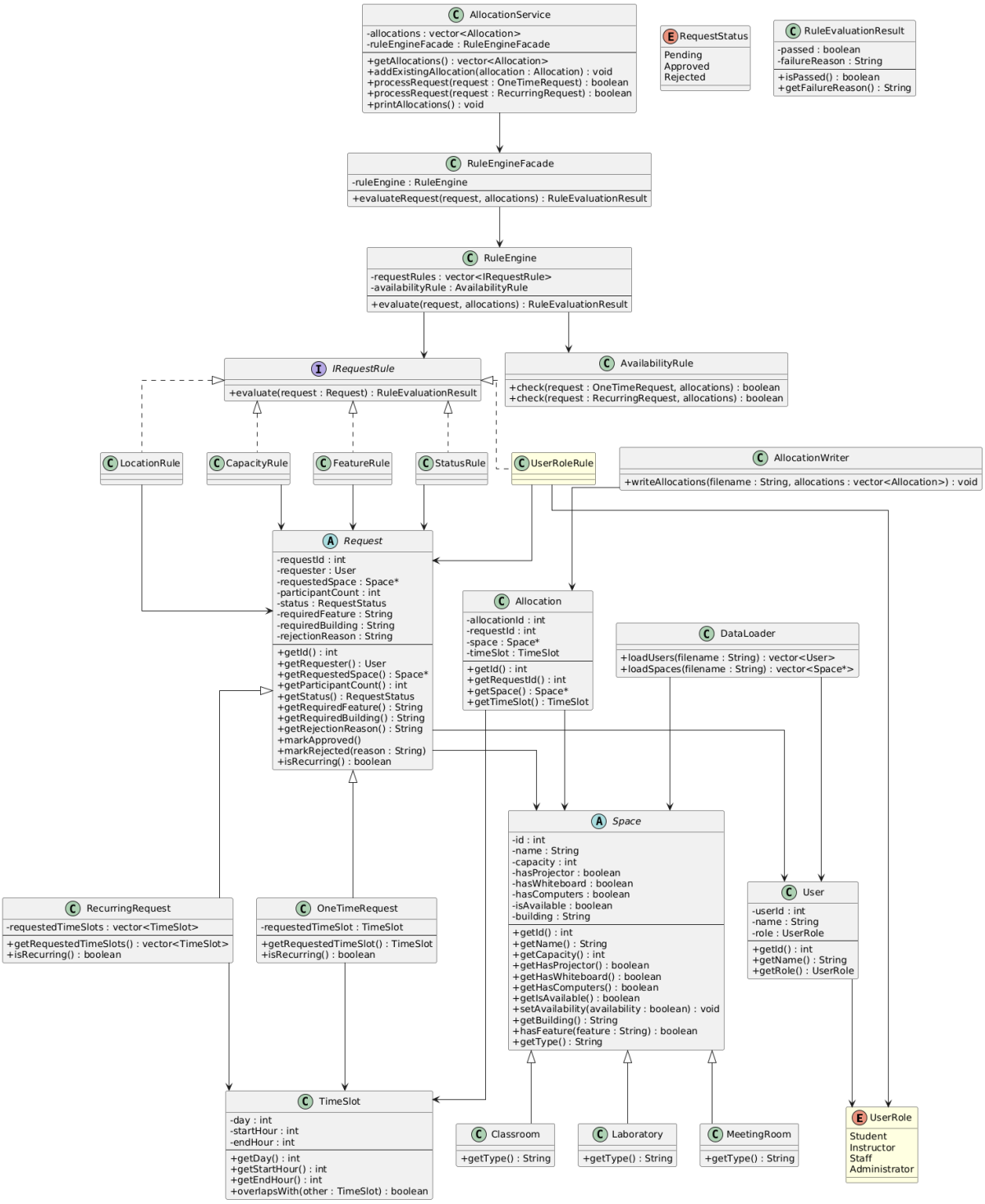

Figure 1: Updated Simplified Class Diagram

8. Updated Workflow Diagram

Figure 2 shows the updated workflow for this iteration. The workflow now includes the additional role-based authorization step inside the rule evaluation process.

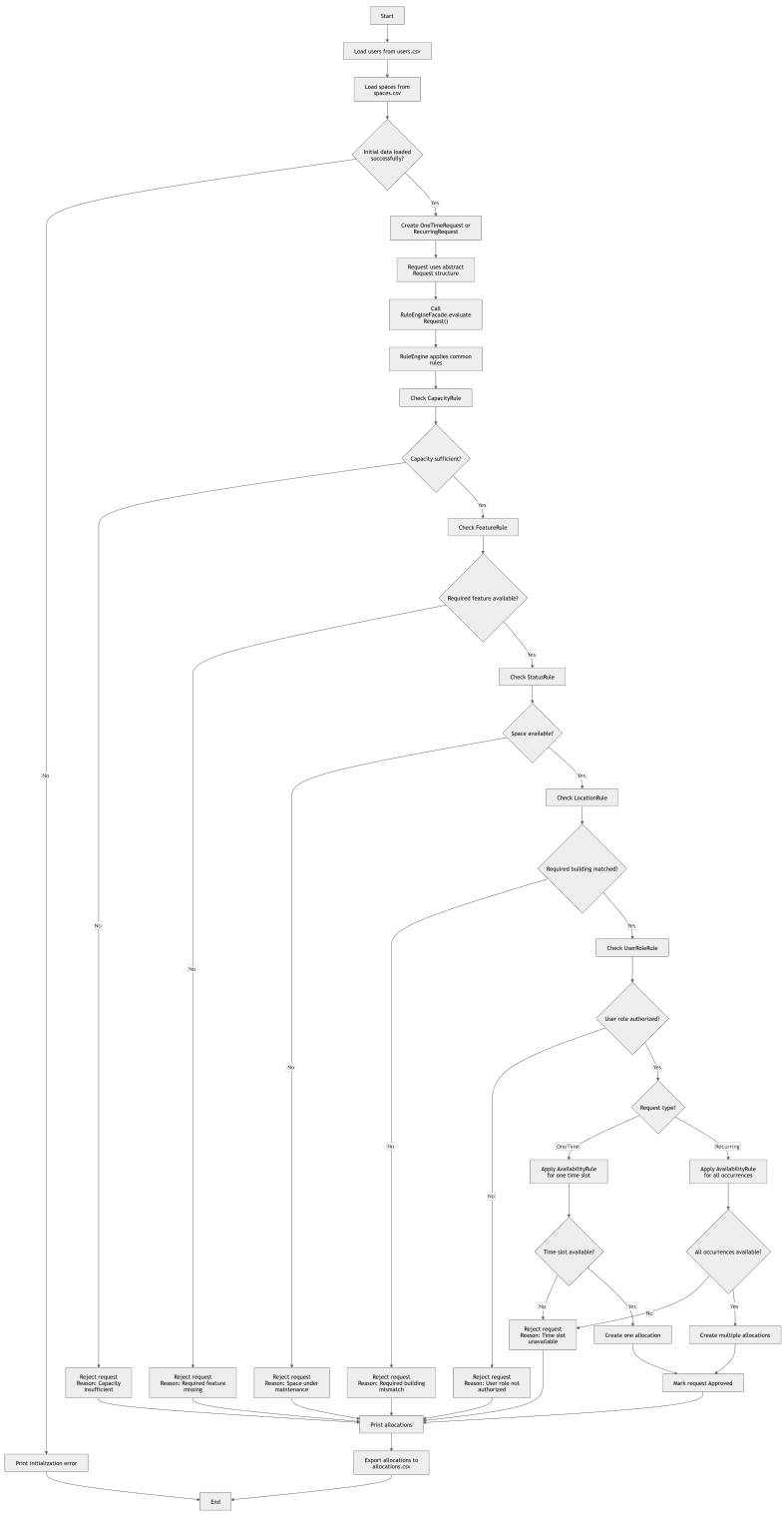

Figure 2: Updated Workflow Diagram

9. Prototype Implementation

The C++ prototype was extended by introducing a new `UserRole` enumeration and updating the `User` class to use it instead of a plain string role. The CSV data loading logic was also updated to parse role values into the structured enum representation.

A new rule, `UserRoleRule`, was added to the rule engine. This rule checks whether the requester's role is authorized for the requested space type. The rule engine facade was updated to include this rule as part of normal request evaluation.

The main test scenarios were updated to include requests from multiple user roles, including students, instructors, staff members, and administrators. This allowed the prototype to demonstrate both successful and rejected requests based on role authorization.

10. Summary

This fourteenth implementation iteration improves the system by making user roles explicit and behaviorally meaningful. Instead of treating the requester role as simple text, the prototype now uses a structured role model and a dedicated authorization rule.

This iteration strengthens both the design and the behavior of the prototype. It also prepares the system for future extensions such as role-based priorities, special permissions, or administrative overrides.